

Contents

1. From Truth Tables to Recursion Tables	1
2. Truth Tables	3
3. Types, Constructors, And Cases	4
4. A First Example: Equality On Natural Numbers	6
Coverage	7
Structure	7
Recursive direction	7
5. Addition And Multiplication	9
6. From Recursion To Induction And Algorithms	11
Lists	12
7. Visibility	13

1. From Truth Tables to Recursion Tables

Truth tables make a small logical structure visible.

Wittgenstein gave them a prominent role in the *Tractatus*, while related tabular representations were also developed by others around the same period. What matters here is not their exact history, but the representational move they make possible.

Consider a logical connective such as conjunction.

Its behavior depends on the truth values of the propositions it connects. Since there are only finitely many combinations of those truth values, they can all be written down.

The resulting table makes the relevant cases surveyable.

Instead of keeping several rules in mind, we can look at one object and inspect them together.

This essay proposes a similar device for a different setting.

Suppose values are constructed from a finite number of constructors, and functions on those values are defined recursively.

Instead of listing combinations of truth values, we can list combinations of constructors.

Instead of putting only final values into the cells, some cells may contain recursive expressions referring to structurally smaller inputs.

We will call such representations **recursion tables**.

The analogy with truth tables is limited but useful.

Truth tables make finite semantic cases easy to inspect.

Recursion tables aim to make two features of simple recursive definitions surveyable:

1. whether all constructor cases have been handled;
2. whether recursive calls move toward structurally simpler cases.

2. Truth Tables

Consider conjunction:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

In truth-functional semantics, the behavior of conjunction is determined by the truth values of its inputs.

There are four possible combinations. The table contains all of them.

If both propositions are true, their conjunction is true.

Otherwise it is false.

With respect to the truth-functional cases represented here, the table makes the relevant possibilities explicit.

This gives it a useful kind of completeness.

We do not have to remember that perhaps another combination exists somewhere outside the table. Given two inputs that each have exactly two possible truth values, the four rows exhaust the combinations.

The usefulness of the table depends on something established beforehand:

- each proposition receives one of the two truth values;
- the listed cases exhaust those possibilities.

Once that background is accepted, the table provides a compact representation of the whole case structure.

Recursion tables begin from the same idea.

But instead of asking which values an expression can receive, we ask how its inputs can be constructed.

3. Types, Constructors, And Cases

Consider the natural numbers.

Instead of beginning with an already completed collection $0, 1, 2, 3, \dots$ we can use the familiar constructive description of the natural numbers.

There are two constructors:

- `zero` is a natural number;
- if `n` is a natural number, then `suc(n)` is a natural number.

Starting from `zero`, repeated use of `suc` gives:

```
zero
suc(zero)
suc(suc(zero))
suc(suc(suc(zero)))
...
```

This constructive presentation is central to approaches such as Martin-Löf type theory, which develops ideas connected to intuitionism, type theory, and the propositions-as-types tradition.

Every natural number constructed in this way has one of two forms.

It is either

`zero`

or

`suc(k)`

for some smaller natural number `k`.

That gives us a finite set of cases even though there are infinitely many natural numbers.

A function on natural numbers can be defined by considering these forms.

For a function of one natural-number argument, there are two cases:

`zero`
`suc(k)`

For a function of two natural-number arguments, there are four combinations:

`zero, zero`
`zero, suc(1)`
`suc(k), zero`
`suc(k), suc(1)`

This is where a table becomes possible.

Recursive calls can also move to structurally smaller data.

If a recursive definition handles `suc(k)` by making a recursive call on `k`, the recursive call is made on structurally smaller data.

For example: $f(\text{suc}(k)) \mapsto f(k)$

removes one `suc`.

Starting with $f(\text{suc}(\text{suc}(\text{suc}(\text{zero}))))$ we move to $f(\text{suc}(\text{suc}(\text{zero})))$, then $f(\text{suc}(\text{zero}))$, then $f(\text{zero})$.

A structurally recursive definition has two features worth watching:

Every constructor case must be handled.

Recursive calls must move toward smaller inputs.

A recursion table can make both visible.

4. A First Example: Equality On Natural Numbers

Consider equality on natural numbers.

The function should return `true` when two natural numbers are the same and `false` otherwise.

Since each input is either `zero` or `suc(...)`, we begin with the four constructor combinations:

=	<code>zero</code>	<code>suc(1)</code>
<code>zero</code>	<code>?</code>	<code>?</code>
<code>suc(k)</code>	<code>?</code>	<code>?</code>

Three cases are immediate.

`zero` equals `zero`.

`zero` does not equal a successor.

A successor does not equal `zero`.

So:

=	<code>zero</code>	<code>suc(1)</code>
<code>zero</code>	<code>true</code>	<code>false</code>
<code>suc(k)</code>	<code>false</code>	<code>?</code>

One case remains.

Two successors are equal exactly when what comes before their outer `suc` constructors is equal:

$$suc(k) = suc(l) \mapsto k = l.$$

The completed table is:

=	zero	suc(1)
zero	true	false
suc(k)	false	k = 1

The table makes three things especially easy to inspect.

Coverage

Every constructor combination appears.

There are two constructors for the first argument and two for the second.

The resulting four combinations all have entries.

This shows that no constructor case has simply been omitted.

Structure

The rows and columns are not arbitrary classifications imposed on the numbers afterwards.

They come directly from the way natural numbers are constructed.

Each input has one of the forms represented by the headers.

The table mirrors the structure of the type.

Recursive direction

Only one entry is recursive:

$$suc(k) = suc(l) \mapsto k = l.$$

The recursive comparison is performed on **k** and **1**, each structurally smaller than the input from which it came.

If both numbers continue to be successors, the same reduction can be repeated.

Eventually at least one side reaches **zero**.

The table does not establish termination merely by being a table.

What it does is put two relevant features in plain view:

- all constructor combinations are covered;
- the only recursive call is made on smaller arguments.

In this simple setting, these are precisely the kinds of features we inspect when checking that evaluation proceeds toward a base case.

But case coverage alone is not enough.

Consider this table:

f	zero	suc(k)
result	0	f(suc(k))

Both constructor cases have entries.

Yet the second entry merely calls the function again on the same input.

It does not move toward **zero**.

If we evaluate

$$f(suc(zero)),$$

we obtain

$$f(suc(zero)),$$

again and again.

The table makes the problem visible precisely because coverage and recursive progress occupy separate places in the representation.

A recursive definition can satisfy the first without satisfying the second.

5. Addition And Multiplication

The same representation can be used for other functions.

Consider addition.

There are several possible recursive definitions of addition. We will use one that reduces the first argument.

The table is:

+	zero	suc(1)
zero	zero	suc(1)
suc(k)	suc(k)	suc(k + suc(1))

The base cases are straightforward.

Adding **zero** to **zero** gives **zero**.

Adding **zero** to **suc(1)** gives **suc(1)**.

Adding **suc(k)** to **zero** gives **suc(k)**.

The final cell contains the recursive case:

$$\text{suc}(k) + \text{suc}(l) \mapsto \text{suc}(k + \text{suc}(l)).$$

The recursive addition inside the outer **suc** has first argument **k**.

The original first argument was **suc(k)**.

So one constructor has been removed.

Consider:

$$2 + 2.$$

Written using constructors:

$$\text{suc}(\text{suc}(\text{zero})) + \text{suc}(\text{suc}(\text{zero})).$$

Applying the recursive case gives:

$$\text{suc}(\text{suc}(\text{zero}) + \text{suc}(\text{suc}(\text{zero}))).$$

Applying it again:

$$\text{suc}(\text{suc}(\text{zero} + \text{suc}(\text{suc}(\text{zero}))))).$$

The base case now applies:

$$\text{suc}(\text{suc}(\text{suc}(\text{suc}(\text{zero}))))).$$

That is 4.

The table places the base cases and the recursive direction in one representation.

Multiplication can be defined similarly:

*	zero	suc(1)
zero	zero	zero
suc(k)	zero	(k x suc(1)) + suc(1)

If either argument is **zero**, the result is **zero**.

The recursive case is:

$$\text{suc}(k) \times \text{suc}(l) \mapsto (k \times \text{suc}(l)) + \text{suc}(l).$$

Again, the multiplication on the right has a smaller first argument.

The original problem contained **suc(k)**.

The recursive multiplication contains **k**.

The definition reduces the multiplication structurally while using addition to construct the result.

6. From Recursion To Induction And Algorithms

Recursive definitions and proofs by structural induction have a closely related shape.

Recursion defines an object or function by following the constructors of its inputs.

Structural induction proves a property by following the same constructor structure.

For natural numbers, an inductive proof typically has two parts:

- establish the property for **zero**;
- establish that, if it holds for **k**, it also holds for **suc(k)**.

This resembles a recursive definition because both follow the same way natural numbers are built.

Recursion tables can make that common structure easier to notice.

Consider the statement:

$$n + \text{zero} = n.$$

For the particular definition of addition used above, the relevant column is already:

+	zero
zero	zero
suc(k)	suc(k)

There are only two possible constructor forms for **n**.

When **n** is **zero**, adding **zero** returns **zero**.

When **n** is **suc(k)**, adding **zero** returns **suc(k)**.

For this definition, no additional inductive argument is required: the two possible forms of **n** are already represented directly by the defining clauses.

Many properties of recursively defined functions are not themselves part of the definition.

A recursion table may show how the function handles each constructor case and where its recursive calls go.

But if we want to prove some further property of the function, we may still need an inductive argument.

The table can help by making the relevant constructor structure visible, but it does not supply that proof by itself.

The table can nevertheless help by showing the constructor structure that such arguments will have to respect.

The same idea extends naturally to algorithms on other inductive data.

Lists

A finite list can be presented using two constructors:

- the empty list **[]**;
- a list formed by attaching an element **x** to another list **xs**, written **x :: xs**.

The length function can then be written as:

$$\text{len}([]) = 0$$

and

$$\text{len}(x :: xs) = 1 + \text{len}(xs).$$

As a table:

len	[]	x :: xs
result	0	1 + len(xs)

There are two constructor cases and both are covered.

The recursive case calls **len** on **xs**.

Since **xs** is the tail from which **x :: xs** was constructed, it is structurally smaller.

The same pattern appears in many recursive programs involving lists, trees, syntax trees, expressions, and other inductively constructed data.

7. Visibility

Recursion tables externalize some of the structure we would otherwise have to keep track of while reading a recursive definition.

For simple examples, two questions become especially visible:

Have all constructor cases been handled?

and:

When recursion occurs, does it move toward structurally smaller data?

Base cases and recursive cases become visually distinguishable.

Recursive reduction can be compared directly with the structure represented in the row and column headers.

This can reduce part of the cognitive burden of understanding a definition.

A recursion table does not by itself show that a definition is correct or that every relevant assumption has been made explicit.

This is what recursion tables are meant to add.

They may be useful pedagogically because they give a reader something compact to inspect before moving to more formal reasoning.